PatchWatch: An Automated Dependency Vulnerability and Update Monitoring System

Project Proposal for UCS503P
Submitted to: Paramveer Sidhu
Thapar Institute of Engineering and Technology

Diya Gupta, CSED* Mysha Bansal, CSED† Durva Garg, CSED‡

August 16, 2026

1 Higher-order goal

. . . secondary framing

This project aims to improve software supply-chain hygiene by reducing the time and effort developers spend tracking, evaluating, and applying dependency updates. While the underlying problem (dependency drift and unpatched vulnerabilities) is broad, the immediate engineering objective is to translate *actionable dependency risk awareness* into a measurable, deployable software solution.

2 Time-to-value

. . . primary heuristic

- We will deliver meaningful increments quickly by prototyping the core scan-and-alert workflow (scan a repository, detect outdated/vulnerable dependencies, surface them) and validating it against real open-source repositories within a short iteration window.
- We will prioritize fast feedback over perfection by using weekly iterations (and targeting CI-backed automation early) to reduce release friction.
- Success is defined by what can be delivered and measured in the first iteration – a working scanner and dashboard – then improved based on feedback from test repositories and users.

3 Problem Statement

Developers maintaining real-world projects struggle to keep dependencies both up to date and secure. Common pain points include:

- **Fragmentation:** dependency health information is scattered across registries (npm, PyPI, Maven), vulnerability databases (OSV, NVD, GitHub Advisory Database), and changelogs, with no single actionable view.
- **Low signal-to-noise:** generic "newer version available" tools flag every update equally, causing alert fatigue and making developers ignore genuinely critical security patches.

*Roll No: 1024030004
†Roll No: 1024030010
‡Roll No: 1024030007

- **No project-specific impact analysis:** existing tools rarely tell a developer *whether their own code* actually uses the affected API/behavior, or which files would need changes.
- **Slow remediation:** even when an update is known to be needed, manually editing manifests, verifying, and testing the change takes time that delays patching.

Why this matters now (contextual relevance): Supply-chain attacks and dependency-based vulnerabilities (e.g., prototype pollution, deserialization flaws) are an increasingly common attack vector; a lightweight tool that turns a firehose of registry/vulnerability data into a prioritized, project-aware action list can meaningfully reduce the window of exposure.

4 Proposed Solution *. . . Software Proposition*

4.1 Overview

Build a **web-based** "PatchWatch" system — a lightweight, Dependabot-style tool that watches a project's dependencies, detects new releases and security patches, checks whether they matter to the project, and tells the developer what to update and why. Core components:

- **Dependency scanner:** parses manifest files (`package.json`, `requirements.txt`, `pom.xml`) to extract current dependency versions.
- **Version monitor:** periodically checks package registries (npm, PyPI, Maven) for newer releases.
- **Security intelligence:** cross-references dependencies against vulnerability databases (OSV, NVD, GitHub Advisory Database).
- **Update classification:** labels each available update as patch/minor/major and flags security-relevant ones.
- **Impact analysis:** performs a lightweight static scan of the codebase to identify which files actually use the affected package/API, so alerts are project-specific rather than generic.
- **Dashboard and GitHub integration:** presents a prioritized view of dependency risk and can open a pull request for a recommended, low-risk update.

4.2 Core workflow

1. Developer connects a GitHub repository; PatchWatch scans manifest files and builds a dependency list.
2. PatchWatch checks each dependency against the relevant package registry (new version?) and vulnerability databases (known CVE/advisory affecting the current version?).
3. The update analyzer classifies each available update (patch/minor/major) and determines security severity.
4. The impact analyzer greps/parses the codebase to find files that import or call the affected package, estimating project impact and required migration effort.
5. PatchWatch produces a ranked dashboard entry and, optionally, opens an automated pull request with the recommended version bump and a summary of why the change is safe.

4.3 Operational constraints for an educational, capstone setting

- Initially support a small set of ecosystems (JavaScript/npm, Python/pip, Java/Maven) rather than trying to cover every package manager.
- Avoid dependence on advanced research algorithms (no ML); focus on registry/vulnerability-DB integration, classification heuristics, and lightweight static analysis.
- Design for deployability using common web hosting, a managed database, and background job workers for periodic scans.

5 Solution Approach *... Engineering focus*

This is an engineering course project, so we focus on scalable data-integration and workflow aspects rather than novel algorithm research.

5.1 Update classification and impact analysis *... non-research*

Classification and impact detection will be heuristic-based, not ML-based:

- Use semantic-version parsing to classify an update as patch, minor, or major.
- Cross-reference the current version range against vulnerability database entries (OSV/NVD/GitHub Advisory) to flag security-relevant updates and assign a severity (low/medium/high/critical).
- Use simple static analysis (import/require statements, Tree-sitter or language ASTs) to list files referencing the affected package, without claiming complete or state-of-the-art code understanding.
- Always present recommendations to the developer for review; never auto-merge without explicit approval.

5.2 Web architecture *... web-based solution*

A typical 3-tier web architecture with an added background-processing layer:

- **Frontend:** responsive dashboard for viewing dependency risk, drilling into a specific package, and triggering PR creation.
- **Backend API:** authentication (GitHub OAuth), repository management, scan orchestration, classification/impact endpoints.
- **Background jobs:** scheduled workers that poll package registries and vulnerability databases and re-scan repositories.
- **Database:** stores repositories, dependency snapshots, scan history, vulnerability matches, and PR status.

5.3 CI/CD and fast delivery enabler

CI/CD will be used to:

- Automatically build and run tests on every change to the PatchWatch codebase itself.
- Produce repeatable deployment artifacts to staging.
- Enable safe and frequent releases of incremental features (new ecosystem support, new vulnerability sources, etc.).

6 Evaluation Criterion *... measurable and attributable*

We define success using measurable metrics tied to the core scan-detect-alert workflow.

6.1 Primary evaluation metric

Time-to-detection (TTD): time between a security advisory/patch being published for a dependency and PatchWatch surfacing an alert for a repository that uses the affected version.

- Target: median TTD $\leq$ 24 hours for repositories under active scanning during the pilot window.
- Attribution: measured from advisory publish timestamp (per OSV/NVD/GitHub Advisory) vs. PatchWatch alert-creation timestamp.

6.2 Secondary evaluation metrics

- **Alert precision:** percentage of security alerts that are relevant (i.e., the flagged package/API is actually used in the codebase), reducing noise compared to naive "new version available" tools.
- **PR acceptance rate:** percentage of automatically generated update pull requests that pass CI and are merged without manual edits, on pilot test repositories.
- **Coverage:** number/percentage of dependencies across supported ecosystems successfully scanned and classified per repository.
- **Reliability:** availability of the scanning service and dashboard during business hours (e.g., $\geq$ 99% uptime in pilot).

6.3 Pilot validation plan *... high-level*

- Connect PatchWatch to 10–15 pilot repositories (a mix of team projects and public open-source repositories with known historical vulnerabilities).
- Compare PatchWatch's detection timing and alert quality against manually checking registries/advisories for the same repositories over the iteration window.

7 Scalability *... with foresight*

The proposition should scale in both theoretical and practical senses.

1. **Scalable (theoretically):** The system should support growth in number of monitored repositories and dependencies by:
   - decoupling scanning/analysis workers from the API and dashboard,
   - queuing and rate-limiting registry/vulnerability-DB requests (e.g., via Redis-backed job queues),
   - enabling pagination and indexing for common dashboard queries.
2. **Operationally deployable (practically):** The system should run on common web hosting:
   - managed relational database,
   - containerized or platform-hosted deployment,
   - CI/CD pipeline for staging/production.

8 Engine availability heuristic

A mature set of "engines" (tooling/services) is available to implement this as a web-based project:

- Web framework for REST APIs and a reactive dashboard (Node.js/Express backend, React/TypeScript frontend).
- Managed relational database (PostgreSQL) with an ORM (Prisma).
- Background job queue for scheduled scans (Redis + BullMQ).
- Public registries and databases: npm/PyPI/Maven registry APIs, OSV API, NVD, GitHub Advisory Database.
- GitHub REST API and OAuth for repository access and PR creation.
- Lightweight code-analysis tooling (Tree-sitter / language ASTs) for impact detection.
- Test framework for unit/integration tests; CI runner and staging deployment target (e.g., Vercel + Render/Railway, Docker).

We will use these mature components to focus on data integration, classification logic, and measurement rather than inventing new infrastructure.

9 Project scope and deliverables *... iteration-friendly*

9.1 Initial deliverable *... first iteration*

- Dependency scanner for one ecosystem (npm/`package.json`) extracting current versions.
- Version monitor querying the npm registry for latest versions.
- Basic dashboard listing dependencies with current vs. latest version and simple patch/minor/major classification.
- Basic logging and timestamps for TTD measurement.
- CI: build + backend unit tests + linting.
- CD to staging with a single command or pipeline job.

9.2 Subsequent deliverables

- Security intelligence integration (OSV/NVD/GitHub Advisory Database) with severity scoring.
- Impact analysis: static-analysis pass identifying files/usages affected by a given dependency update.
- GitHub OAuth integration, repository scanning, and automatic pull-request creation for recommended updates.
- Support for additional ecosystems (Python/`requirements.txt`, Java/`pom.xml`).
- Notification layer and richer admin dashboard (backlog of unresolved security issues, historical trends).

10 Risks and mitigations

- **Third-party API rate limits:** registry and vulnerability-DB APIs may throttle requests; mitigate with caching, batching, and a background job queue rather than synchronous on-demand calls.
- **False positives/negatives in impact analysis:** static analysis may miss dynamic usages or over-flag files; mitigate by presenting impact analysis as an aid for developer review, not an automatic gate.
- **Data staleness:** vulnerability databases and registries update independently; mitigate with a defined re-scan interval and clear "last checked" timestamps.
- **Team adoption of auto-generated PRs:** developers may distrust automated updates; mitigate by keeping PRs low-risk by default (patch/security-only), always running tests, and never auto-merging.

11 Summary

This project proposes PatchWatch, a deployable web platform that reduces the time and effort required to detect, prioritize, and remediate outdated or vulnerable project dependencies. It meets the course criteria by emphasizing time-to-value through rapid iterations, implementing CI/CD to support frequent safe releases, and using measurable evaluation criteria (especially time-to-detection and alert precision). It remains focused on engineering integration, classification, and workflow design rather than research-driven novelty, while still offering a substantial, demonstrable end-to-end system (scanner $\rightarrow$ security intelligence $\rightarrow$ impact analysis $\rightarrow$ dashboard $\rightarrow$ automated PR).